Lab 1: HTTP Server and Pub-Sub Broker

Please note: You need to implement this yourselves. There are open frameworks like Gin and Martini that you are not allowed to use.

Take a look at the Go programming language if you are not sure how to write Go.

1. HTTP server (Basic part, 10 points)

Your task is to build a web server capable of accepting HTTP requests and returning response data from locally stored files to a client. The server will be implemented in Go and MUST handle **concurrent** requests by creating a Go routine for each new client request. You will only be responsible for implementing the GET and POST methods. All other request methods received by the server should elicit a *"Not Implemented" (501)* error (see **RFC 1945** section 9.5 - Server Error).

Your web server should compile and run without errors or warnings, producing a binary called `http_server` that takes as its first argument a port to listen on. Don't use a hard-coded port number. You shouldn't assume that your server will be running on a particular IP address, or that clients will be coming from a predetermined IP.

Listening

When your server starts, the first thing that it will need to do is establish a socket connection that it can use to listen for incoming connections. Your server should listen on the port specified from the command line and wait for incoming client connections. Each new client request is accepted, and a new Go routine is spawned to handle the request. To avoid overwhelming your server, you should not create more than a reasonable number of child processes (for this assignment, use at most 10). In case an additional child process would break this limit, your server should wait until one of its ongoing child processes exits before forking a new one to handle the new request.

Once a client has connected, the server should read data from the client and then check for a properly-formatted HTTP request. Your server should accept requests for files ending in html, txt, gif, jpeg, jpg, or css and transmit them to the client with a Content-Type of text/html, text/plain, image/gif, image/jpeg, image/jpeg, or text/css, respectively. If the client requests a file with any other extension, the web server must respond with a well-formed *400 "Bad Request"* code. An invalid request from the client should be answered with an appropriate error code, i.e. *"Bad Request" (400)* or *"Not Implemented" (501)* for valid HTTP methods other than GET. If the requested file does not exist, your server should return a well-formed *404 "Not Found"* code. Similarly, if headers are not properly formatted for parsing or any other error condition not listed before, your server should also generate a type-400 message. For POST requests, please make

sure that you store the files and make them accessible with a subsequent GET request. And several clients may send POST requests for the same file at the same time, while other clients are reading it. Your server should make sure that these updates are atomic: a GET request should always return one complete version of the file, not a partially written or mixed one.

Parsing and Networking Libraries in Go

For this assignment, you should use the package `net` for the networking, for example using `net.Listen("tcp", address)` to listen for incoming TCP connections. You can also use the package `net/http`, but ONLY for parsing and working with HTTP request objects, and not the networking part. You should not use e.g., `http.ListenAndServe` which trivializes the assignment (the same goes for `http.Listen` and `http.Serve`).

2. Pub-sub broker (Bonus part, up to 7 extra bonus points):

The publish-subscribe pattern lets clients receive messages on a topic without knowing who produces them or when. Publishers send messages on a topic to a broker, and the broker forwards each message to every subscriber of that topic. Publishers and subscribers never communicate directly, and they do not need to know about each other.

Your task is to extend your HTTP server from Part 1 into a simple pub-sub broker. The broker must run in the same `http_server` binary and on the same port as the file server, and it must still follow the networking rules of Part 1.

Operations

Your broker should support the following four operations.

(1) `POST /subscribe?topic=X` registers a new subscriber on topic X, and responds 201 "Created" with the subscriber's id, e.g., {"id":N}. The subscriber uses this id in later requests.

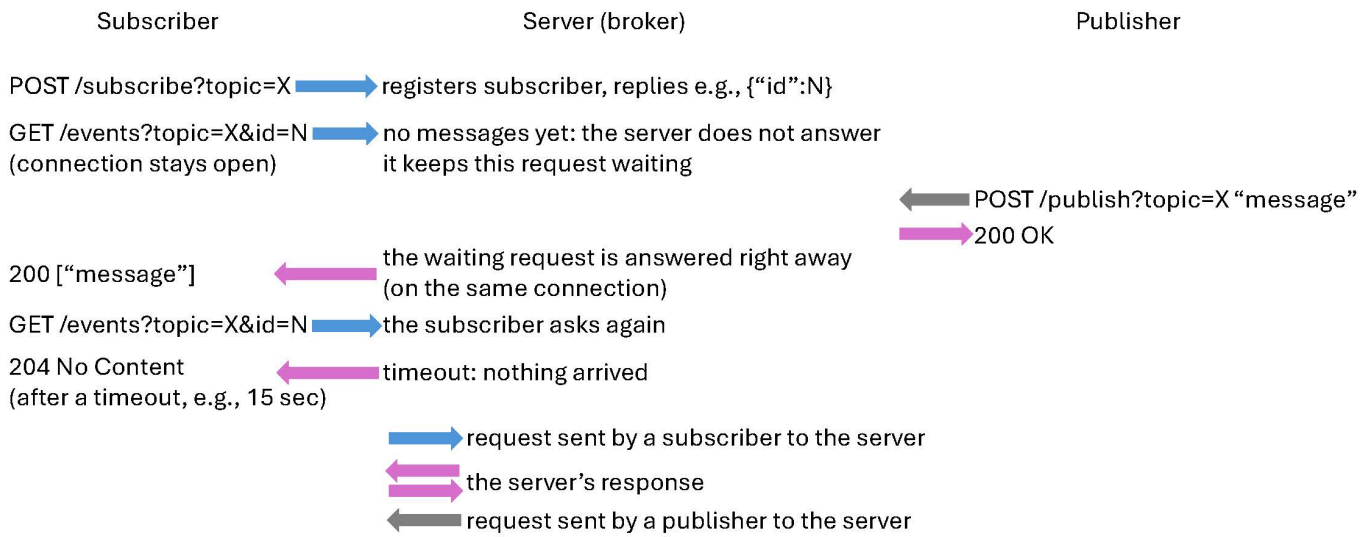
(2) `POST /unsubscribe?topic=X&id=N` removes subscriber N.

(3) `POST /publish?topic=X` sends the request body as a message to every current subscriber of X.

(4) `GET /events?topic=X&id=N` delivers subscriber N's messages, using (a) long polling: the server holds the request until a message arrives or a timeout expires, and the subscriber then polls again, and it responds *204 "No Content"* if nothing arrives within the timeout; or (b) streaming: the server keeps one response open and writes each message to it as it arrives; or (c) any other approach that delivers a message to a waiting subscriber as soon as it is published, as long as you can explain your choice.

Requests with a missing topic or invalid id should return *400 "Bad Request"*, requests for an unknown subscriber *404 "Not Found"*, and other methods *501 "Not Implemented"*.

The figure below shows an example of how a subscriber receives a message with long polling. Note that all requests are sent by the clients; the broker only responds to them, and the message is delivered in the response to the subscriber's waiting GET /events request.



Concurrency

Every request runs in its own Go routine, so the broker's state (topics, subscribers and their queues) is shared and must be protected; you could keep it simple, such as using a single mutex for the whole broker. Your broker should handle concurrent publishers and subscribers, so that each message gets a sequence number and every subscriber receives the messages of its topic in that order. Subscribers may subscribe and unsubscribe at any time, and only receive messages published while they are subscribed. Give each subscriber a bounded queue (e.g. a buffered channel), and make sure publishing never blocks: if a subscriber's queue is full, it is fine to simply skip that subscriber for that message. Long pollings that are waiting for messages may be excluded from the limit of 10 concurrent Go routines from Part 1, since they are not doing any work while they wait.

Strongly recommended: use containers to help you build this and reuse it later for further labs

Look at either Docker, LXC or Vagrant to implement your server. This way you will never have portability issues, and it will make your solution cleaner. Bonus for this will also be considered based on the cleanliness of your code/solution.

3. Cloud (up to 3 extra bonus points)

Move to the Cloud!

Access to AWS: you can all use 50$ worth of credits for the AWS platform via **AWS Learner Lab**. All students are invited to join a Canvas course on their official Chalmers/GU email at the beginning of the course. The invitation email comes from "AWS Academy

<notifications@instructure.com>" to join **AWS Canvas**. When you click on "Get Started", choose "Create an Account" -- This is because AWS Canvas is different from Chalmers Canvas. Check your spam and if you're still having trouble accessing AWS, contact the examiner.

4. Testing

There are no included tests for this assignment. However, to make sure your code works, you should set up a way to test its functionality yourself.

5. Grading

Each group will have 30 minutes to present their solution, during which we will ask you questions about your code and run tests on your implementation. Make sure to prepare the demo before your turn for the assessment.

Lab responsible: Jingyu Liu <jingyu.liu@chalmers.se> and Yixing Zhang <yixing@chalmers.se>